Monad 编排:模式与协议设计(第一稿)

> 状态:**第一稿,等待评审**。2026-10-06。 配套文档:monad-orchestration.md(政策与设计目标)。 本文聚焦"怎么做":trait 形状、async 拓扑、handoff 协议、状态机。 **不在本文范围**:具体 monad 实现(计算 / 库存 / 采购等留给各自实现)、持久化(P5)、UI 展示。

---

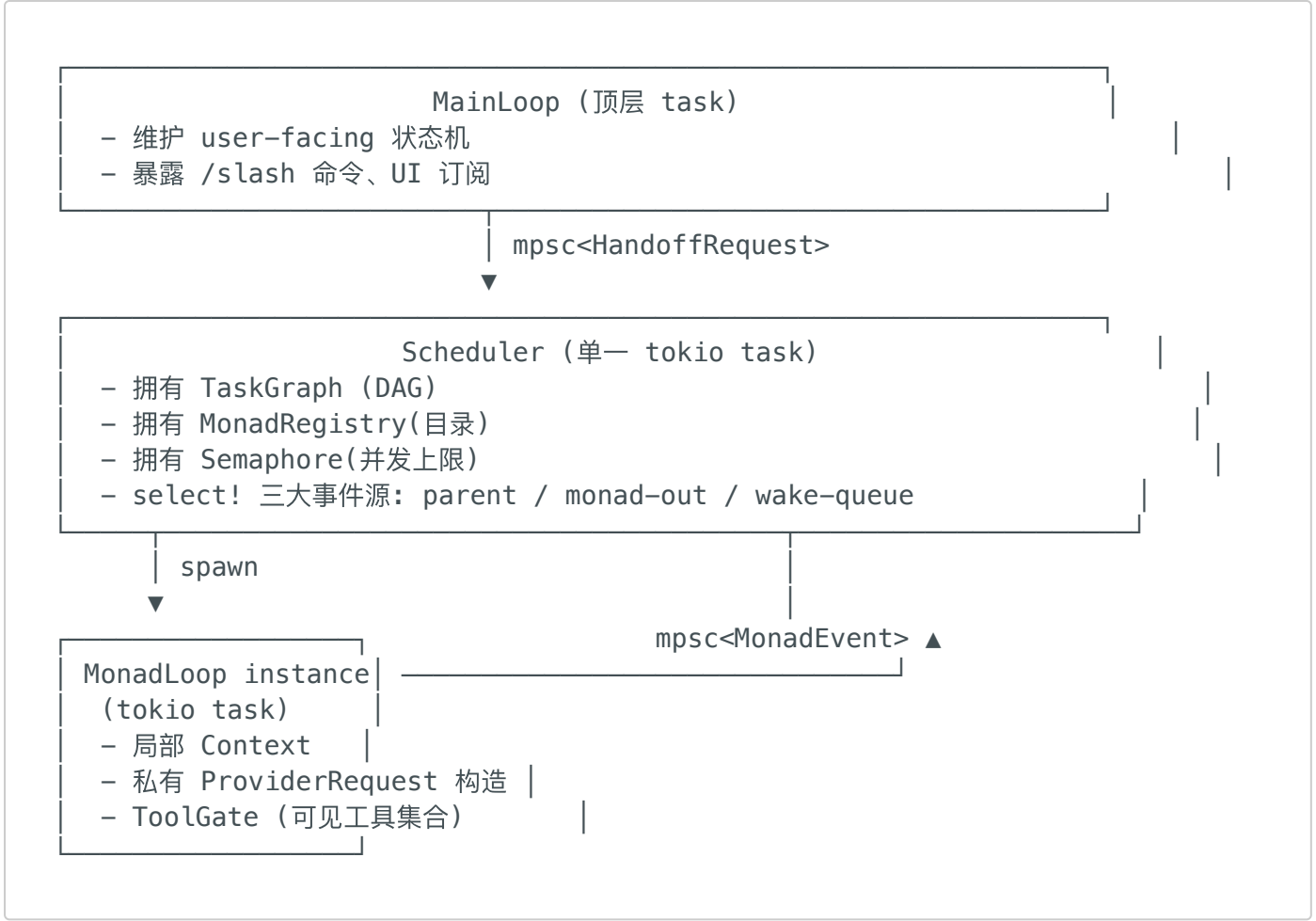
0. 文档定位

`monad-orchestration.md` 回答 **why** 与 **what**(为什么分层、什么场景能验收)。 本文档回答 **how**(具体怎么落地)。

阅读顺序建议:先 monad-orchestration §1–§3,然后回本文。

---

1. 类型全景与拓扑



关键约束:

- Scheduler **唯一**(单点),所有调度逻辑集中。避免分散在多个 task 里导致状态不一致。
- 每个 MonadLoop 跑在自己的 task 里,**互不共享可变状态**。跨 monad 通信只通过 Scheduler。
- Semaphore 由 Scheduler 持有,任何 MonadLoop 想启动资源动作(网络/写)都要 acquire 许可。

---

2. 核心 trait 草案

2.1 MonadLoop — 每个 monad 必须实现的契约

```rust
#[async_trait]
pub trait MonadLoop: Send + Sync + 'static {
    /// 注册时唯一。后续 TaskGraph / ResultSlot 都引用此 id。
    fn monad_id(&self) -> &MonadId;

    /// 短摘要(供主循环目录)。不应包含任何方法细节。
    fn catalog_entry(&self) -> &CatalogEntry;
```

```
    /// 此 monad 默认可见的工具集合。
    /// 主循环不直接看到这些工具,只有被 handoff 后才在该 monad 的 frame 内可见。
    fn default_tool_set(&self) -> Vec<ToolKind>;

    /// 接收 handoff,做局部初始化(读 inputs、构造初始消息、注入约束),
    /// 然后发起首轮 ProviderRequest,返回 InitialFrame。
    async fn start(
        &mut self,
        handoff: MonadHandoff,
        ctx: MonadContext,    // 含 ToolGate, Provider handle, Budget
    ) -> Result<InitialFrame, MonadError>;

    /// 上一轮 tool_exec 完毕后,再走一轮。
    /// 若已完成则返回 Final,否则返回 Next。
    async fn step(
        &mut self,
        prev: StepInput,      // 含 tool_execution_results + 上轮 usage
    ) -> Result<StepOutput, MonadError>;

    /// 硬性取消 hook。允许清理资源(关连接、撤回 inflight 请求)。
    async fn cancel(&mut self, reason: CancelReason) -> Result<(),
MonadError>;
}
```

设计抉择:

- `start` 与 `step` 拆开,不用一个 `tick(state, event)`。原因是 `start` 拿到的是完整 handoff,`step` 拿到的是增量事件——两者语义不同。
- `tool_set` 是 `default`,实际可见集合由 `MonadContext.tool_gate` 决定,允许 main-frame 临时收窄(比如禁止写操作)。
- `cancel` 是 best-effort,失败不 panic。

2.2 `MonadRegistry` — 目录与实例化

```
pub trait MonadRegistry: Send + Sync {
    /// 主循环可见。仅返回短摘要。
    fn list(&self) -> Vec<CatalogEntry>;

    /// 按 monad_id 取一个 *新实例*。
    /// Scheduler 在派发时调用,避免多个任务复用同一实例。
    fn instantiate(&self, id: &MonadId) -> Result<Arc<dyn MonadLoop>,
RegistryError>;

    /// 按 monad_id + 关键词过滤(留给 P4 大量 monad 时使用)。
    fn search(&self, query: &str, limit: usize) -> Vec<CatalogEntry>;
}
```

第一版实现 可以是 `Vec<Box<dyn MonadFactory>>`,每个 factory 提供 monad_id、catalog_entry 和 `instantiate()`。

2.3 `ToolGate` — 运行时检查

```rust
pub struct ToolGate {
    allowed: HashMap<ToolKind, ToolPermissions>,
}

impl ToolGate {
    /// 在构造 ProviderRequest 前调用,过滤掉当前 frame 不该出现的工具。
    pub fn filter(&self, tools: &[ToolKind]) -> Vec<ToolSchema>;

    /// 在执行 tool_call 前调用。返回拒绝原因。
    pub fn check(&self, call: &ToolCall) -> Result<(), GateError>;
}
```

`ToolGate` 不是 trait——它是 **具体类型**,由 Scheduler 在 spawn MonadLoop 时根据 handoff.constraints 构造。

---

3. Handoff 协议

3.1 JSON Schema(草案)

> **注意**:这是 wire format,Rust 类型只是镜像。

```jsonc
{
  "schema_version": "monad-handoff/v1",

  // 标识与归属
  "task_id":          "uuid-v7-string",        // 本次 handoff 的 id
  "parent_task_id":   "uuid-v7-string | null",
  "monad_id":         "calculation",           // 目标 monad
  "attempt_id":       1,                        // 重试计数,首次 1

  // 任务语义
  "objective":        "计算 100 加 100",          // 主任务原话
  "inputs":           { "expression": "100+100" },
  "relevant_context": [
    { "kind": "user_quote", "text": "先算 100 加 100" },
    { "kind": "prev_result", "task_id": "...", "summary": "上一步结果" }
  ],
  "constraints":      [ "expression 必须合法", "返回整数" ],
  "completion_criteria": "返回数值结果及原始表达式",
```

```
  "result_slot":     "calculations.first",   // 父任务图中的槽位

  // 调度依赖
  "depends_on":      [ "task-id-A" ],          // 依赖的其他 task_id

  // 资源预算
  "budget": {
    "max_steps":        6,
    "timeout_ms":       30000,
    "max_tokens":       4000,
    "concurrent_io":    1
  },

  // 元数据
  "issued_at_ms":    1700000000000,
  "trace": {
    "frame_ref": "frame-rev-xxx",  // 父 RequestFrame 的版本
    "request_id": "req-yyy"
  }
}
```

3.2 Rust 类型

```rust
#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(rename_all = "snake_case")]
pub struct MonadHandoff {
    pub schema_version: SchemaVersion,             // "monad-handoff/v1"

    pub task_id: TaskId,
    pub parent_task_id: Option<TaskId>,
    pub monad_id: MonadId,
    pub attempt_id: u32,

    pub objective: String,
    pub inputs: serde_json::Map<String, serde_json::Value>,
    pub relevant_context: Vec<ContextRef>,
    pub constraints: Vec<String>,
    pub completion_criteria: String,
    pub result_slot: ResultSlot,

    #[serde(default)]
    pub depends_on: Vec<TaskId>,

    pub budget: Budget,
    pub issued_at_ms: i64,
    pub trace: TraceMeta,
}

#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(tag = "kind", rename_all = "snake_case")]
pub enum ContextRef {
    UserQuote { text: String },
    PrevResult { task_id: TaskId, summary: String },
    MemoryHit { source: MemoryTable, ref_id: String, summary: String },
```

```
    // 后续 P4 加: External { url, snippet }, Skill { name, version },
}

#[derive(Debug, Clone, Serialize, Deserialize)]
pub struct Budget {
    pub max_steps: u32,
    pub timeout_ms: u64,
    #[serde(default = "default_max_tokens")]
    pub max_tokens: u32,
    #[serde(default = "default_concurrent_io")]
    pub concurrent_io: u32,
}
```

3.3 版本与兼容性

- `schema_version` 字段强制存在,反序列化时若不匹配当前 `SchemaVersion` 枚举:
  - **major 不匹配** → 拒绝,要求升级
  - **minor 不匹配** → 接受,缺失字段走默认值
- `inputs` 用 `serde_json::Map` 而非固定 struct,**允许每个 monad 自定义** schema。第一版不强类型,留给 monad 自己 validate。

3.4 Result(回填)协议

```
#[derive(Debug, Clone, Serialize, Deserialize)]
#[serde(rename_all = "snake_case", tag = "status")]
pub enum MonadResult {
    Completed {
        task_id: TaskId,
        parent_task_id: Option<TaskId>,
        result_slot: ResultSlot,
        monad_id: MonadId,
        output: serde_json::Value,
        evidence_refs: Vec<String>,    // 引用 execution_id 等
        usage: ResourceUsage,
        finished_at_ms: i64,
    },
    WaitingContext {
        task_id: TaskId,
        reason: String,                // 缺什么字段
        missing: Vec<String>,          // 字段名列表
    },
    Failed {
        task_id: TaskId,
        error: MonadError,
        retryable: bool,
    },
    Cancelled {
```

```
        task_id: TaskId,
        reason: String,
    },
}
```

`task_id / parent_task_id / result_slot` 三元组唯一锁定结果回填位置——**不靠事件到达顺序**。

---

4. Async 拓扑与消息流

4.1 Channel 划分

Channel	方向	装的内容	缓冲
`parent_to_sched`	main → scheduler	`HandoffRequest`(外部)	bounded(64)
`sched_to_monad`	scheduler → monad	`MonadHandoff`(启动信号)	bounded(4)
`monad_to_sched`	monad → scheduler	`MonadResult`	bounded(64)
`wake_queue`	scheduler 内部	`TaskId`(唤醒被阻塞任务)	unbounded
`event_bus`	scheduler → main	`SchedulerEvent`(回填)	bounded(64)

为什么不用 broadcast:我们不要"全部订阅者都收到",每条事件有明确的去向(task graph → 目标 task)。 **为什么 wake_queue unbounded**:短暂阻塞恢复时不应丢消息。但要在持久化层留 hook 限长(P5)。

4.2 Scheduler 主循环

伪代码:

```
async fn scheduler_loop(
    mut parent_rx: mpsc::Receiver<HandoffRequest>,
    mut monad_rx:   mpsc::Receiver<(TaskId, MonadResult)>,
    registry:       Arc<dyn MonadRegistry>,
    semaphore:      Arc<Semaphore>,
) {
    let mut task_graph: TaskGraph = TaskGraph::new();
    let mut running:    HashMap<TaskId, JoinHandle> = HashMap::new();
```

```
    let mut wake:       VecDeque<TaskId> = VecDeque::new();

    loop {
        tokio::select! {
            Some(req) = parent_rx.recv() => {
                // 外部新任务 → 加节点 → 入 ready
                let node = task_graph.add(req);
                if task_graph.deps_satisfied(node) {
                    dispatch(&mut task_graph, &mut running, node, &registry,
&semaphore).await;
                } else {
                    task_graph.mark_blocked(node);
                }
            }
            Some((task_id, result)) = monad_rx.recv() => {
                // monad 回结果 → 更新 graph → 唤醒后续
                handle_result(&mut task_graph, &mut running, &mut wake,
task_id, result).await;
                // 尝试派发被 wake 的任务
                while let Some(id) = wake.pop_front() {
                    if task_graph.try_make_ready(id) {
                        dispatch(&mut task_graph, &mut running, id,
&registry, &semaphore).await;
                    }
                }
            }
        }
    }
}
```

`dispatch` 内部:

1. `semaphore.clone().acquire_owned().await` —— 拿并发许可
2. `registry.instantiate(monad_id)` —— 新实例
3. `tokio::spawn(monad_task(...))` —— 独立 task
4. 持有 `JoinHandle` + 释放 semaphore 的 RAII guard

4.3 取消传播

```
MainLoop.cancel(parent_task_id)
   ↓ scheduler
遍历 task_graph 所有 child
   ↓ 每个 child MonadLoop
monad.cancel(CancelReason::ParentCancelled)
   ↓ 异步 join JoinHandle + abort
清理 task_graph 节点
```

不去做的:取消正在执行的 Provider HTTP 请求(网络层硬中断留给 P5)。第一版只取消本地 task 编排。

5. 状态机

5.1 状态枚举

```rust
#[derive(Debug, Clone, Copy, PartialEq, Eq, Serialize, Deserialize)]
#[serde(rename_all = "snake_case")]
pub enum TaskState {
    Pending,            // 已创建,依赖未满足
    Ready,              // 依赖满足,等待 dispatch
    Running,            // 已 spawn,monad 执行中
    WaitingContext,     // monad 返回 needs_context
    Blocked,            // 依赖任务尚未完成
    Completed,          // 收到 Completed result
    Failed,             // 收到 Failed result (retryable=false 或超过 attempt)
}
```

注意:`Cancelled` 不作为独立状态——cancelled 任务从 graph 中**移除**,不再跟踪。

5.2 转换表

From	Event	To
(none)	add	`Pending`
`Pending`	all deps completed	`Ready`
`Ready`	dispatch	`Running`
`Running`	result = Completed	`Completed`
`Running`	result = WaitingContext	`WaitingContext`
`Running`	result = Failed (retryable, attempt < max)	`Pending` (new attempt)
`Running`	result = Failed (non-retryable)	`Failed`
`Running`	parent cancelled / timeout	`Failed` (Cancelled reason)
`WaitingContext`	parent supplies context	`Ready`
`WaitingContext`	timeout	`Failed`

5.3 持久化要点(P5 之前先 in-memory)

第一版所有 TaskState 都活在 `TaskGraph` 内存里,重启即丢。 P5 要:

- `task_graph` 序列化到 SQLite / LanceDB
- 启动时 replay,`running` 状态标 `Blocked`(已重启,等外部查询接续)
- `attempt_id` 持久,防止重启后重做副作用

---

6. 与现有 `agent_loop.rs` 的衔接

小项目的 `run_one_turn_with_progress` 当前是"内联"的 main loop。 第一阶段改造路径(分 PR,每改对应 monad-orchestration.md §11 的阶段):

6.1 P1 阶段:不破坏现有,加一层适配

不动 `agent_loop.rs` 内部,把它的"主循环"包成一个**内置 MainMonad**:

```
pub struct MainMonad {
    inner: AgentLoop,           // 现成的,提供 run_one_turn_with_progress
}

impl MonadLoop for MainMonad {
    fn monad_id(&self) -> &MonadId { &MAIN_MONAD_ID }
    fn catalog_entry(&self) -> &CatalogEntry { /* "main" 短摘要 */ }
    fn default_tool_set(&self) -> Vec<ToolKind> { /* 全量 tools */ }

    async fn start(&mut self, handoff, ctx) -> Result<InitialFrame,
MonadError> {
        // 把 handoff.objective 当 user message 注入 AgentLoop.state.messages
        // 用现有 progress stream 作为 InitialFrame
    }
    // ...
}
```

这一步让"主循环也走 MonadRunner 协议",后续 P2/P3 加子 monad 时不用改 main。

6.2 P2 阶段:加单层子 monad

让 MainMonad 在合适时机(检测到某个工具调用或业务关键词)**生成 MonadHandoff** 发给 Scheduler,接收 MonadResult 后**回填到当前 message stream** 作为 tool_result。

这一阶段**不并发**,只验证 handoff 协议 + 局部 context 隔离。

6.3 P3+:真正的并发与画布追踪

在 P1+P2 协议稳定后才动。

---

7. 待评审点(我自己的疑虑)

7.1 关于 `MonadLoop` trait 的颗粒度

- 当前 `start`/`step` 拆开。**也可**合成一个 `tick(event)` + 状态机外部驱动。
- 替代方案:把 `MonadLoop` 做成"事件源",Scheduler 推 `Handoff | ToolResult | Cancel`,MonadLoop 输出 `Step | Final | NeedsContext`。**这更 reactive**。
- 我倾向当前方案(显式生命周期),但如果你更 reactive 偏好,可改。

7.2 关于 `inputs: serde_json::Map`

- 灵活,但**没有类型安全**。
- 替代:每个 monad 注册自己的 `Inputs: DeserializeOwned`,由 Registry 在 instantiate 时解码。
- 第一版先 Map,P1 跑稳再考虑类型化。

7.3 关于"主循环 + 业务循环"是不是两层 while

文档 §3 用"主循环 / 业务循环"的说法,但实际可能是:

- 一个 main task (含 scheduler)
- 多个 monad tasks **没有显式的 "while"**。 如果你读 §3 时觉得"两层 while"误导,我可以**改文档用语为"主任务 / 子任务"**。

7.4 关于 ToolGate 是类型还是 trait

当前做成类型(由 Scheduler 构造)。但 monad 可能希望自定义 Gate 规则。

- 替代:`pub trait ToolGate: Send + Sync`,每个 monad 可自带 impl。
- 我倾向"框架默认实现 + monad 可覆盖",留待 P4 决定。

7.5 关于持久化时 attempt 是否自增

`Failed(retryable=true, attempt=2)` 是否应该:

- (a) 在 Scheduler 重 spawn 时 attempt++
- (b) 让 MonadLoop 自己决定 attempt 是否累加
- 我倾向 (a)——框架管 attempt,monad 管语义,职责清楚。

7.6 关于 cancel 与 abort 的语义

当前 `cancel()` 是 best-effort hook。问题是:

- 如果 monad 在 tokio::select! 里等外部响应,`cancel` 调用没法中断它。
- 真正的中断要 abort JoinHandle + 让 monad 检测 "is_cancelled" flag。
- 第一版建议:**Scheduler 持有 cancel token,monad 自己每次 await 前 select!{ _ = cancel.cancelled() }**。

---

8. 与 `monad-orchestration.md` 的映射

政策文档 §	本模式文档 §
§1 要解决的问题	§1 拓扑(给具体形状)
§3 模块与职责	§2 trait 草案
§4.4 RequestFrame / 画布	§2.1 `MonadContext`、§6.2 MainMonad 包装
§5.1 发现	§2.2 `MonadRegistry.list`
§5.2 交接	§3 协议
§5.3 执行	§4 async 拓扑
§5.4 回填	§3.4 Result 协议
§6 协作	§4.2 Scheduler 主循环
§7 并发	§4.2 Semaphore、§5 状态机

政策文档 §	本模式文档 §
§8 失败/恢复	§4.3 取消、§5.3 持久化要点
§10 可观察性	留待单独 doc(本轮不展开)
§11 阶段	§6 衔接
§12 实验	留待实验 doc

---

9. 评审反馈收集

读完请回答(或自由补充):

1. **trait 形状** —— §2 的 `start/step/cancel` 拆,合你的胃口吗?还是 `tick(event)` 更合?
2. **JSON schema** —— §3.1 的字段够不够,有没有想加/想删?
3. **主循环包成 MainMonad** —— §6.1 这个"零破坏"路径 OK 吗?
4. **取消语义** —— §7.6 的"cancel token + monad 主动 select!" 接受吗?
5. **最大疑虑** —— §7 里哪一条你最想先讨论?

我会按你的反馈出第二稿。